64. Minimum Path Sum - Detailed Explanation

🔥 Problem Statement

Aapko ek `m x n` grid di gayi hai jisme non-negative numbers (positive numbers aur zero) bhare hue hain.

Aapka task ek path dhundhna hai **Top-Left (0,0)** se lekar **Bottom-Right (m-1, n-1)** tak.

Lekin ek shart hai: Aap kisi bhi waqt pe sirf **Down (Neeche)** ya **Right (Daayein)** move kar sakte hain.

Humein ek aisa path dhundhna hai jiske raaste me aane wale sabhi numbers ka sum sabse **minimum** (kam se kam) ho.

Example se samjho:

Input grid:

```
[1, 3, 1]
[1, 5, 1]
[4, 2, 1]
```

Agar aap ye path follow karein: `1 (0,0) → 3 (0,1) → 1 (0,2) → 1 (1,2) → 1 (2,2)`

Matlab: Right → Right → Down → Down.

Toh inka sum hoga: `1 + 3 + 1 + 1 + 1 = 7` . Ekdum minimum!

🧠 Brute Force Approach (Recursion - Kyun TLE dega?)

Har cell `(i, j)` se hamare paas 2 choices hain:

1. Ya toh right jao `(i, j+1)`

2. Ya toh down jao `(i+1, j)`

Toh hum sabhi possible paths ka sum nikal sakte hain recursion use karke aur usmese minimum evaluate kar sakte hain.

Lekin iski Time Complexity bahut zyada ho jayegi, kareeb `O(2^(m+n))` kyunki har point par `2` choices hain aur overlapping subproblems banenge (ek hi cell pe alag paths se pahuchenge aur usko baar baar solve karenge). Is wajah se isme **Time Limit Exceeded (TLE)** aayega.

🚀 Optimized Approach (Dynamic Programming - Memoization)

Is problem ko fast solve karne ke liye hum **Dynamic Programming (Top-Down / Memoization)** use karenge.

Jab recursion se overlapping subproblems aate hain, toh hum unke results ek array ya matrix me store kar lete hain taaki baar-baar calculate na karna pade.

DP State ka matlab:

Hum ek `dp[i][j]` matrix banayenge jahan:

`dp[i][j] = (i, j)` cell se lekar `(m-1, n-1)` aakhiri cell tak jaane ka minimum sum.

Base Cases:

1. **Out of Bound:** Agar `i ⩾ m` ya `j ⩾ n` ho jayega toh wo un-reachable rasta hai, isiliye yaha se bahut badi value return karo (jaise `1e9`) taaki `min()` logic me ye count na ho aage.
2. **Destination reached:** Agar hum exactly `(m-1, n-1)` par khade hain toh rasta wahi khatam ho gaya, return kardo waha ka grid element `grid[m-1][n-1]` .

Transition Logic:

Hume `(i, j)` ka answer chahiye, toh ye kisse depend karega?

`dp[i][j] = current cell value + min( right jane ka score , down jane ka score )`

```
dp[i][j] = grid[i][j] + min( f(i, j+1), f(i+1, j) )
```

Memoization addition:

Function evaluate karne se pehle check karo ki kya `dp[i][j]` pehle calculate ho chuka hai (yani `-1` nahi hai). Agar calculate ho rakha hai toh wahi se value direct return kardo, repetitive recursion ka time bach jayega!

💻 Code Architecture Recap

1. Ek global `m` aur `n` variables function definition ko aasaan banayenge taaki size constraints easily check hongi.
2. `dp` table 2D matrix li aur sari values `-1` se initialize kardi, yaha constraints me max limit dekhi toh row & col size 200 hain toh `vector<vector<int>> dp(200, vector<int>(200, -1))` properly array limit fix ki.
3. Helper function `f(i, j)` banaya jo recursion aur memoization use karega in states ko traverse karne ko.
4. Finally `minPathSum()` me `f` ko `(0, 0)` arguments ke saath call kiya.

Time Complexity: `O(m * n)` kyunki har cell `(i,j)` sirf 1 baar compute hota hai DP ki wajeh se.

Space Complexity: `O(m * n)` table + max stack call recursive calls me `O(m+n)` time pe run ho sakti hai. Total Space is mainly DP table `O(m * n)` .

Hinglish aur logic based aise flow mein DP ka flow directly mind mein chipak jayega! 🤘